

LAB: KỸ THUẬT TẤN CÔNG VÀ PHÒNG THỦ VỚI DLL TRONG WINDOWS

Mục tiêu bài Lab:

- Tự lập trình và xây dựng các loại mã thực thi bên trong thư viện liên kết động (DLL) bằng C++.
- Thực hành kỹ thuật DLL Injection để thao túng tiến trình hệ thống và đánh cắp dữ liệu.
- Xây dựng hệ thống giám sát cấp độ Kernel (Sử dụng Sysmon và PowerShell) để phát hiện hành vi nạp DLL trái phép.

Môi trường thực hành:

- Máy ảo Windows 10/11 (Mục tiêu tiêm DLL & Cài đặt Sysmon).
- Máy ảo Ubuntu (Đóng vai trò C2 Server và Máy biên dịch).
- Công cụ: RemoteDLL, Sysmon (Microsoft Sysinternals), Notepad++.

BƯỚC CHUẨN BỊ: CÀI ĐẶT MÔI TRƯỜNG BIÊN DỊCH (TRÊN UBUNTU)

Để tạo ra file DLL chạy được trên Windows từ máy Ubuntu, sinh viên cần cài đặt trình biên dịch chéo (Cross-compiler) MinGW-w64.

Mở Terminal trên Ubuntu và chạy lệnh sau:

```
sudo apt update  
sudo apt install mingw-w64 -y
```

PHẦN 1: LẬP TRÌNH VÀ DEMO CÁC PHƯƠNG THỨC THỰC THI MÃ TRONG DLL (RED TEAM)

Kịch bản 1.1: Thực thi ngầm qua hàmDllMain (Không Export)

Cơ sở lý thuyết: HàmDllMain là Entry Point (điểm vào) của một file DLL. Khi một phần mềm nạp DLL vào bộ nhớ, hệ điều hành sẽ tự động gọi hàmDllMain. Kẻ tấn công lợi dụng điểm này để chèn mã độc (Payload) vàoDllMain mà không cần khai báo bất kỳ hàm Export nào, giúp che giấu hành vi.

Nhiệm vụ của sinh viên:

1. Tạo một file tên làkb1_no_export.cpptrên Ubuntu.
2. **Yêu cầu lập trình:** * Viết cấu trúc hàmDllMain tiêu chuẩn.
 - Bắt sự kiệnDLL_PROCESS_ATTACH.

- Sử dụng hàm WinExec (hoặc system) của thư viện <windows.h> để gọi ứng dụng máy tính (calc.exe) chạy ngầm, hoặc mở một lệnh ping.
- Tuyệt đối không sử dụng từ khóa `__declspec(dllexport)`.

Biên dịch và Thực thi:

1. Biên dịch file C++ thành DLL trên Ubuntu:
`x86_64-w64-mingw32-g++ -shared -o kb1.dll kb1_no_export.cpp`
2. Chuyển file kb1.dll sang Windows.
3. Mở CMD trên Windows, sử dụng công cụ mặc định rundll32.exe để ép thực thi:
`rundll32.exe kb1.dll,Test`

(Sẽ có thông báo lỗi do không tìm thấy hàm Test, nhưng hãy kiểm tra Task Manager xem Payload có được kích hoạt ngầm hay không).

Kịch bản 1.2: Thực thi thông qua hàm Export

Cơ sở lý thuyết: Đôi khi mã độc cần sự điều khiển có chủ đích thay vì chạy ngay lúc nạp. Payload sẽ được đặt trong một hàm Export (được khai báo rõ ràng). DllMain lúc này hoàn toàn "sạch" để qua mặt các hệ thống Sandbox quét tự động.

Nhiệm vụ của sinh viên:

1. Tạo file kb2_export.cpp trên Ubuntu.
2. **Yêu cầu lập trình:**
 - Viết hàm DllMain chỉ trả về TRUE (Không chứa payload).
 - Viết một hàm tên là LaunchWeb. Khai báo hàm này là Export function bằng từ khóa: `extern "C" __declspec(dllexport)`.
 - Bên trong hàm LaunchWeb, sử dụng hàm ShellExecuteA (thư viện <shellapi.h>) để mở trình duyệt web và truy cập vào một trang web bất kỳ.

Biên dịch và Thực thi:

1. Biên dịch trên Ubuntu:
`x86_64-w64-mingw32-g++ -shared -o kb2.dll kb2_export.cpp`
2. Chuyển kb2.dll sang Windows và gọi đích danh hàm Export:
`rundll32.exe kb2.dll,LaunchWeb`
3. Quan sát trình duyệt tự động bật lên.

Kịch bản 1.3: DLL Injection & Đánh cắp dữ liệu (Data Exfiltration)

Cơ sở lý thuyết: Kỹ thuật tiêm DLL (DLL Injection) ép một tiến trình hợp pháp (Ví dụ: Notepad++.exe) phải công file DLL của Hacker. Khi đã ở trong tiến trình nạn nhân, DLL có thể đọc trộm bộ nhớ, giao diện và gửi dữ liệu ra ngoài Internet.

Nhiệm vụ của sinh viên:

1. **Viết C2 Server (Ubuntu):** Tạo file server.py, dùng thư viện socket của Python để lắng nghe TCP ở port 8080 và in ra dữ liệu nhận được. Chạy server: python3 server.py.
2. **Lập trình Malware DLL (Ubuntu):** Tạo file kb3_injection.cpp.
 - o **Yêu cầu:** Khởi tạo một luồng mới (CreateThread) bên trongDllMain.
 - o Luồng này sẽ gọi các hàm WinAPI (FindWindowW, EnumChildWindows, SendMessageW với tham số WM_GETTEXT) để đọc trộm đoạn văn bản đang được gõ trên Notepad++.
 - o Sử dụng thư viện Winsock2 (<winsock2.h>) để tạo socket, kết nối về IP của máy ảo Ubuntu (C2 Server) ở port 8080 và gửi dữ liệu văn bản vừa trộm được.

Biên dịch và Tấn công:

1. Biên dịch trên Ubuntu (Lưu ý: Phải link thư viện mạng ws2_32):
x86_64-w64-mingw32-g++ -shared -o malware.dll kb3_injection.cpp -lws2_32
2. Chuyển malware.dll sang Windows. Mở phần mềm **Notepad++**.
3. Khởi động công cụ **RemoteDLL** (Run as Administrator).
4. Chọn target là tiến trình Notepad++.exe, chọn đường dẫn tới file malware.dll và nhấn **Inject DLL**.
5. Nhập văn bản bất kỳ vào Notepad++ và quan sát màn hình Ubuntu xem dữ liệu có bị đánh cắp hay không.

PHẦN 2: DETECT VIỆC NẠP DLL VÀO MỘT PROCESS (BLUE TEAM)

Cơ sở lý thuyết: Các phần mềm diệt virus thông thường (quét đĩa tĩnh) sẽ không thể bắt được file DLL độc hại nếu nó sử dụng kỹ thuật mã hóa (Obfuscation) hoặc tàng hình trong RAM (Reflective Injection).

Để phát hiện, Blue Team phải can thiệp vào tầng lõi hệ điều hành (**Kernel-Mode**) sử dụng **Kernel Callbacks**. Bất cứ khi nào có hành vi nạp một Image (DLL/EXE) vào RAM, Kernel sẽ báo cáo. Trong Lab này, chúng ta dùng **Sysmon (Event ID 7: Image Loaded)** làm "Thiên nhãn".

Bước 2.1: Triển khai Kernel Driver (Sysmon)

1. Cài đặt công cụ **Sysmon** của Microsoft trên Windows.
2. Tạo file cấu hình edr_rules.xml để yêu cầu Sysmon chỉ giám sát các DLL được nạp vào tiến trình nhạy cảm (Notepad++.exe) và bỏ qua các DLL hợp lệ của Windows:
<Sysmon schemaversion="4.82">
 <EventFiltering>
 <RuleGroup name="Detect DLL Injection" groupRelation="or">
 <ImageLoad onmatch="include">
 <Image condition="end with">notepad++.exe</Image>
 </RuleGroup>
 </EventFiltering>
 </Sysmon>

```

    </ImageLoad>
</RuleGroup>
<RuleGroup name="Exclude Legitimate DLLs" groupRelation="or">
    <ImageLoad onmatch="exclude">
        <ImageLoaded condition="begin with">C:\Windows\</ImageLoaded>
        <ImageLoaded condition="begin with">C:\Program Files\</ImageLoaded>
    </ImageLoad>
</RuleGroup>
</EventFiltering>
</Sysmon>

```

3. Mở **CMD (Quyền Administrator)** và nạp luật cho Kernel:
`.\Sysmon64.exe -i edr_rules.xml`

Bước 2.2: Xây dựng màn hình Radar (SOC Console)

Chúng ta sẽ dùng PowerShell để kết nối vào luồng dữ liệu của Sysmon và hiện cảnh báo thời gian thực.

1. Tạo file `mini_soc.ps1` và sao chép đoạn mã sau vào:
 # Đặt kích thước và màu sắc cửa sổ cho ngẫu
`$host.ui.RawUI.WindowTitle = "BLUE TEAM RADAR - KERNEL MODE MONITORING (TẦNG 2)"`
`Clear-Host`
`Write-Host`
`"=====`
`-ForegroundColor Cyan`
`Write-Host "[*] HE THONG MINI-EDR DA KHOI DONG" -ForegroundColor Green`
`Write-Host "[*] Dang ket noi voi Kernel Driver (Sysmon)..." -ForegroundColor Yellow`
`Write-Host "[*] Dang lang nghe su kien Nap DLL (ImageLoad - Event ID 7) vào`
`notepad++.exe" -ForegroundColor Yellow`
`Write-Host`
`"=====`
`-ForegroundColor Cyan`
`Write-Host "Dang cho tin hieu tu Kernel... (An Ctrl+C de thoat)`n"`

`# Lấy thời điểm bắt đầu chạy script để không đọc lại log cũ`
`$startTime = Get-Date`

`while ($true) {`
 `$events = $null`

 `try {`

```

# Truy vấn Event Viewer lấy log Sysmon từ thời điểm script chạy
$events = Get-WinEvent -FilterHashtable @{
    LogName='Microsoft-Windows-Sysmon/Operational'
    ID=7 # Event ID 7 là Image Loaded
    StartTime=$startTime
} -ErrorAction Stop
}
catch {
    $errMsg = $_.Exception.Message
    # Bỏ qua lỗi thông thường khi chưa có sự kiện nào phát sinh
    if ($errMsg -like "**Parameter is incorrect*" -or $errMsg -like "**No events were found*")
    {
        # Không làm gì cả, cứ lặp tiếp
    }
    else {
        # Báo động nếu Sysmon chưa được cài đặt hoặc cấu hình sai
        Write-Host "`n🚨 [LỖI HỆ THỐNG]: Không thể đọc log Sysmon!" -ForegroundColor
Red
        Write-Host "Lý do: $errMsg" -ForegroundColor DarkGray
        Write-Host "=> BẠN CHƯA CÀI ĐẶT SYSMON HOẶC CHƯA NẠP FILE XML!"
-ForegroundColor Yellow
        Write-Host "=> Hãy mở CMD (Admin) và chạy lệnh: .\Sysmon64.exe -i edr_rules.xml"
-ForegroundColor Yellow
        Start-Sleep -Seconds 5
    }
}

if ($events) {
    foreach ($event in $events) {
        # Ép kiểu XML để đọc dữ liệu chi tiết
        $xml = [xml]$event.ToXml()
        $eventData = $xml.Event.EventData.Data

        $image = ($eventData | Where-Object { $_.Name -eq 'Image' }).'#text'
        $imageLoaded = ($eventData | Where-Object { $_.Name -eq 'ImageLoaded' }).'#text'
        $hashes = ($eventData | Where-Object { $_.Name -eq 'Hashes' }).'#text'

        # Hiển thị cảnh báo đỏ rực trên màn hình
        Write-Host "`a" # Phát tiếng bíp
        Write-Host "🚨 [PHÁT HIỆN TÀN CỐNG!] KERNEL CALLBACK ĐÃ BẮT ĐƯỢC MÀ
DOC 🚨" -ForegroundColor White -BackgroundColor Red
        Write-Host "Thời gian : $($event.TimeCreated)" -ForegroundColor Red
        Write-Host "Tên trình : $image" -ForegroundColor Red
    }
}

```

```

Write-Host "DLL Bị Nạp : $imageLoaded" -ForegroundColor Yellow
Write-Host "Chu ký Hash: $hashes" -ForegroundColor DarkGray
Write-Host "-----`n"
}
# Cập nhật lại thời gian để không in lại log cũ
$startTime = Get-Date
}
# Tạm dừng 1 giây để không làm treo máy
Start-Sleep -Seconds 1
}

```

2. Mở **PowerShell (Quyền Administrator)**.
3. Gỡ bỏ rào cản thực thi Script của Windows (Nếu bị báo lỗi chữ đỏ):
Set-ExecutionPolicy Bypass -Scope Process -Force
4. Khởi động hệ thống Radar:
.\\mini_soc.ps1

Bước 2.3: Thực nghiệm Tấn công & Đánh chặn

1. Sắp xếp màn hình: Một bên để **RemoteDLL**, một bên để màn hình **Blue Team Radar** (PowerShell).
2. Đóng vai Red Team: Dùng RemoteDLL thực hiện lại hành động Inject malware.dll vào Notepad++.exe.
3. Đóng vai Blue Team: Ngay lập tức, màn hình PowerShell sẽ phát tiếng bíp và **chớp đỏ rực**, in ra chính xác tên tiến trình nạn nhân và đường dẫn file DLL độc hại vừa xâm nhập.

Tổng Kết Bài Học (Takeaways)

- Hành vi thực thi của DLL rất đa dạng (DllMain, Export, Injection), gây khó khăn cho việc phân tích tĩnh.
- Việc tự tay lập trình DLL giúp hiểu rõ API của Windows và thấy được ranh giới mong manh giữa phần mềm hợp pháp và mã độc.
- Để phòng thủ hiệu quả các hình thức cấp cao (như Fileless Malware / DLL Injection), hệ thống bắt buộc phải có cơ chế giám sát thời gian thực dựa trên Hành vi (Behavior) và phải được cắm ở tầng sâu của hệ điều hành (Kernel/ETW).